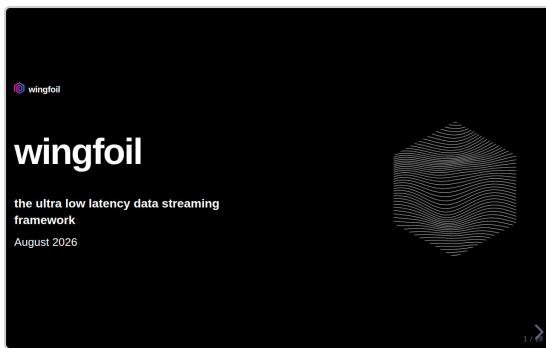


wingfoil — speaker notes

28 slides · printed fallback for the reveal.js speaker view (press S)



1 / 28

UP WHILE YOU ARE INTRODUCED AND THE ROOM SETTLES. Nothing here competes with your voice — that is the point of opening on the logo.

THE SPOKEN OPEN (no slide for this — say it, don't show it):

"Thanks very much for coming tonight. My name is Jake Mitchell, and I'm here to talk about my open source project, wingfoil.

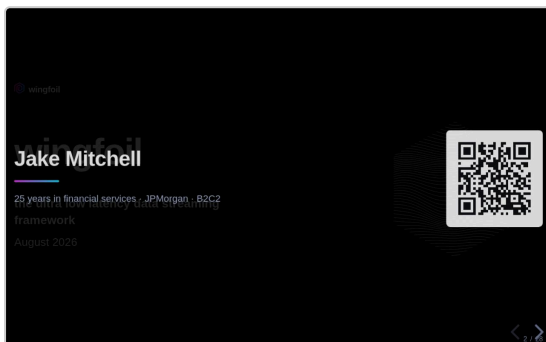
I'm going to do that in three parts.

First, a bit of background — about me, and about the kind of problem wingfoil is for. I'll show you what it is, how you use it, and take you round some of what's in the box.

Second — Nitro. That's the performance story: what we wanted to add, why it meant essentially rewriting the whole framework to get it, the design pattern we uncovered along the way, and what it bought us.

And third, where this goes next — down the stack and up the stack — and where we could really use some help."

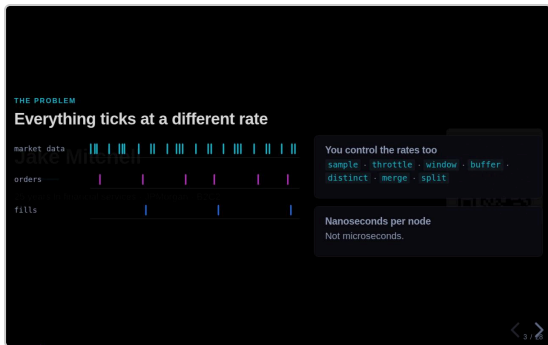
Then advance. Do not linger.



2 / 28

Jake Mitchell

Keep it to 45 seconds. The point of the CV is not the CV — it is that the problems in part one are ones you have actually had, for 25 years, at firms where getting them wrong costs money. Then get off it. The QR is here so the room can scan early rather than scrambling at the end.

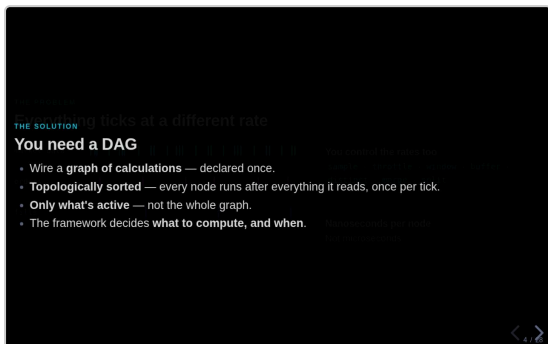


3 / 28

THE PROBLEM

Everything ticks at a different rate

90 seconds. Set up the whole talk here: three feeds, three rates, none of them aligned, and the rate itself is something you need to control rather than something handed to you. Be precise about the grouping, because someone will push back on the word "burst": the point is that many messages carry the SAME exchange timestamp — one aggressive order sweeping several levels is several fills at one instant. That is a property of the data, not of how fast you are reading it. Wingfoil delivers same-timestamp values together and never collapses them to latest-wins, because a strictly-monotonic clock means a split same-time group cannot be reassembled afterwards. Then: performance. Nanoseconds per node. That constraint is what rules out most of the obvious answers, which is the next slide.



4 / 28

THE SOLUTION

You need a DAG

This is the thesis of act one — state it flat, then spend the next two slides earning it. Don't over-explain "doubles every level" here; slide 6 is the demonstration and slide 7 is the chart. The fourth bullet is the one to lean on: the framework's job is deciding what to compute and when. Everything after this slide is how that is actually done. Backup number if challenged on "only compute what is needed": an ~8-node hot path inside a ~1030-node graph costs 984 ns per cycle on the dirty-list scheduler against 6.05 us for a naive full sweep — 6.2x apart. The naive dispatcher is kept as an executable oracle, same results either way.



5 / 28

THE SURFACE

Wire it, build it, run it

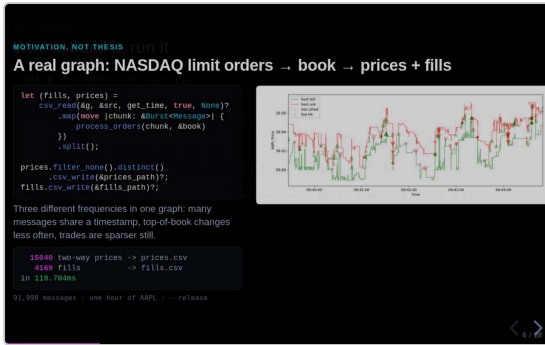
Combinators are extension traits, not inherent methods — come back to that on the extensibility slide. `for\_each` is the outbound edge: values stream out as the run progresses rather than piling into a Vec. Note HistoricalFrom finishes instantly — five 100ms ticks, no waiting. That is the same wiring you would run live.

6 / 28

MOTIVATION, NOT THESIS

A real graph: NASDAQ limit orders → book → prices + fills

This is here to buy credibility for the rest of the talk, not to teach the API. 90 seconds max. The number to say out loud: an hour of real market data, 92 thousand messages, replays in a tenth of a second — and the same wiring runs live against a venue feed.

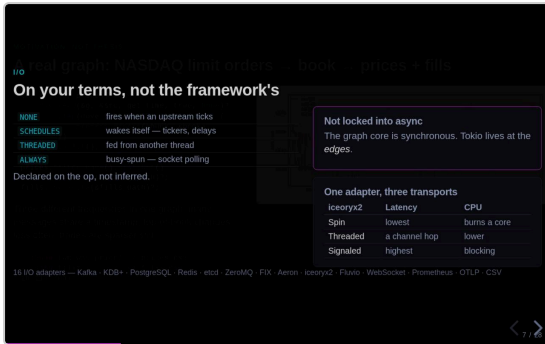


7 / 28

I/O

On your terms, not the framework's

The point of this slide: most frameworks make the concurrency decision FOR you, and it is usually async. Here the scheduling contract is a const on the op, so a busy-spin socket poller and a blocking WaitSet subscriber are both first-class, not workarounds. The iceoryx2 table is the concrete proof — one adapter, three polling modes, and the user picks by passing an argument. 60-75 seconds. If short on time, say the async line and the iceoryx2 table and skip the four-mode table.

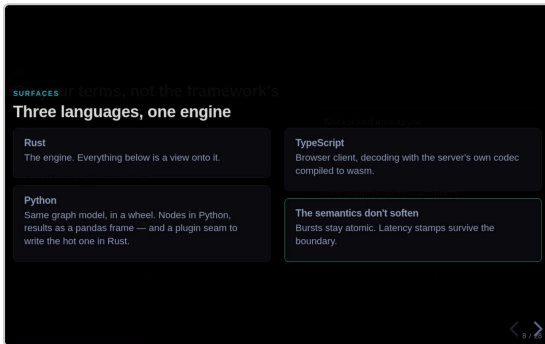


8 / 28

SURFACES

Three languages, one engine

Keep it brisk — 45 seconds. The claim that matters is the last card: the semantics do not soften at a language boundary. A Python user gets bursts and latency tracing, not a simplified model. The plugin seam is the answer to "isn't Python going to be slow?" — you move the hot node to Rust without leaving the graph.

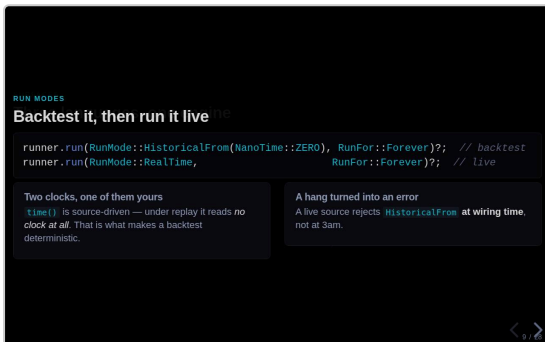


9 / 28

RUN MODES

Backtest it, then run it live

This is the slide most of the audience actually wants, so do not rush it. ~75 seconds. The two-clock split is the whole determinism story: replay reads no clock, so the same input gives the same output every time; the wall snap is taken lazily on first read and cached in a Cell, so a cycle where nothing stamps reads the clock zero times — which matters because that read is ~24 ns. If asked "what about a graph that is partly live and partly replayed" — that is what the rejection above is protecting, and the honest answer is that you run two graphs.



Backtest it, then run it live

PART TWO

Nitro

Two choices, one of them yours. Performance, the wall we hit chasing it, and an enemy and the pattern that got us through.

```
let mut source = g.constant(1_u128);
for _ in 1..128 {
    source = source.join(&source, [a, b] a + b); // branch and recombine
}
```

1 ticks processed in 12.953µs
value 1791411834488469231731687363718884109728

That value is 2^{127} — the correct answer, which is also exactly the number of source → sink paths a framework that propagates one path at a time would have to walk.

10 / 28

PART TWO

Nitro

Transition. You should be around the 9-minute mark. Say the line and move.

THE HOOK

127 levels deep. 2^{127} distinct paths. One tick.

```
let mut source = g.constant(1_u128);
for _ in 1..128 {
    source = source.join(&source, [a, b] a + b); // branch and recombine
}
```

1 ticks processed in 12.953µs
value 1791411834488469231731687363718884109728

That value is 2^{127} — the correct answer, which is also exactly the number of source → sink paths a framework that propagates one path at a time would have to walk.

11 / 28

THE HOOK

127 levels deep. 2127 distinct paths. One tick.

THIS is the slide that wakes the room up. Land it slowly. "Every one of those 2^{127} paths is a real path through the graph. A reactive library that pushes along one path at a time has to walk all of them. We visit 128 nodes." Pause. Then next slide explains why.

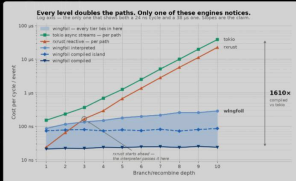
WHY IT STAYS FLAT

Visit each node once per tick

A node runs after everything it reads, exactly once — however many paths lead to it.

Propagate one path at a time and a shared node is re-visited per path. Branch and recombine, and that doubles every level.

Rx + async streams
Measured slopes 2.01x and 1.94x per level.
Wingfoil: flat.



12 / 28

WHY IT STAYS FLAT

Visit each node once per tick

Be scrupulous here — a Rust audience will respect the honesty and punish the lack of it. Say out loud: the rxrust figure is HALVED from the raw ratio because its benchmark emits twice per sample; the tokio target is recursive Box::pin over an immediately-ready leaf, which is an honest depiction of per-path propagation but not the best an async implementation could do. The claim is the SLOPE, not the multiplier.

Visit each node once per tick

THE ENGINEERING PROBLEM

What we wanted: compile the graph

Interpreted, every node pays a dynamic dispatch and a trip through a value slot — per node, per cycle.

On a hot chain that dominates the arithmetic you came to do.

So: generate code. One straight-line function, state in locals, optimiser working across node boundaries.

It failed three ways — and the third told us why.

We built it
Ran the wiring from build.rs, walked the graph, emitted a runner.

13 / 28

THE ENGINEERING PROBLEM

What we wanted: compile the graph

~75 seconds, and sell the plan honestly: it is a good plan, it is what anyone would try, and every ingredient really does exist. That is what makes the next slide land. The failure is not incompetence, it is a design constraint nobody had named yet.

WHY A REWRITE, NOT A FEATURE

The generator failed three ways

- 1 It had to re-implement every node. Semantics were welded to `RefCell` fields, so generated code could not call a node's cycle. Two `map`s, two `filter`s — nothing keeping them in step.
- 2 Types and closures were already gone. Wiring erased them. You cannot get `[x] x * 10` back out of a `Box.`
- 3 Scheduling was inferred from names. Nothing declared "I schedule myself", so the engine drove nodes off allowlists.

Node semantics were not *separately callable*.

The generator failed three ways

The argumentative core of the talk — take your time, roughly 30 seconds a wall. Wall 2 usually gets a laugh: a human re-typing the closure that already exists ten lines up, with no compiler check that the two agree. Then the punch, slowly. No amount of cleverness in the emitter fixes a design where there is nothing to emit a call TO. That sentence is why the engine got rewritten rather than extended.

Inside-out

THE PATTERN

Inside-out

ONE OBJECT OWNS EVERYTHING

Logic
state
input plumbing

called three ways

* nothing outside can call the logic

The tension

Encapsulation says bundle state with behaviour. Single responsibility says one reason to change. Follow the first far enough and you break the second.

INSIDE-OUT

Engine

Cfg · State

Send

Op::cycle

the logic

The resolution

Encapsulate the logic and the types. Let ownership live outside.

Node semantics were not *separately callable*.

Ownership moved out. Encapsulation didn't.

The op owns *what* and *how*. The engine owns *where* and *when*.

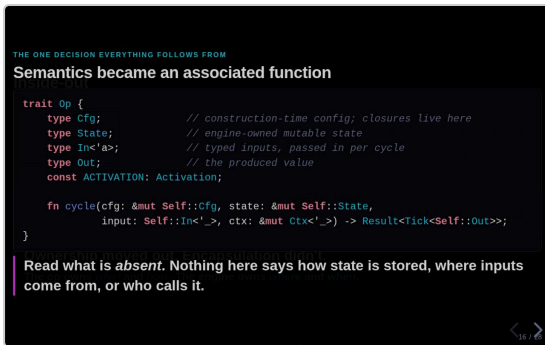
THE closing slide. ~90 seconds, and do not rush the last line. Three beats: the tension (everyone in the room has this problem), the resolution (ownership out, encapsulation in), then the payoff — the op owns what and how, the engine owns where and when — which is the promise made back on slide 3 being paid. Point at the arrow. That arrow IS the pattern: the engine passes state it cannot read into logic it cannot see. Neither side knows the other's interior, and that is why three different engines can drive one definition. Name the cousins OUT LOUD. A Rust room is already thinking "isn't this an ECS?", and getting there first turns a hostile question into agreement. The claim is the combination, not the shape.

Optional, if the room is warm and you want a name they will remember: the Japanese for an inside-out roll — rice outside, nori in — is "uramaki". Deploy it as a throwaway or skip it; nothing on the slide depends on it.

Q&A backup — the three things we would tell someone doing this rewrite, which used to be this slide: 1. Decide the contract questions first, in blast-radius order. Fallibility touched every op; deciding it late means retrofitting every emitter. 2. Point the dependency edge at the new thing — the old tree depended on the new one, so the last step was a deletion, not an archaeological dig. 3. Make the invariant structural, not aspirational. "Keep the paths in sync" is a promise that decays; "the second copy is a compile error" is a property.

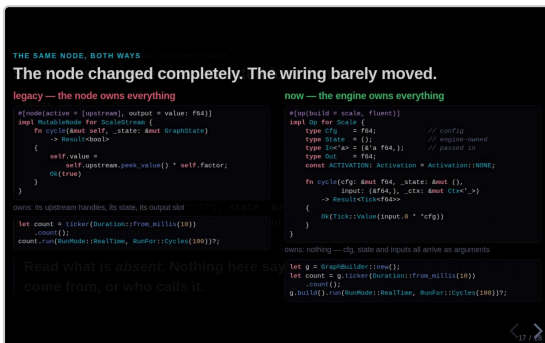
Semantics became an associated function

Not a method on an instantiated object — an associated function over explicit arguments. Every one of those four associated types is a hole the engine fills, and the engine gets to choose HOW it fills them. Don't rush to the payoff — make them look at the absence first. Plant the phrase here, in one sentence, without putting it on the slide: "this is an inside-out object — the logic stays wrapped, the state moves outside." Slide 20 is where that gets its name and its picture, so a room that has heard the phrase once already arrives at the close primed. If asked about the return type: Tick has THREE states, not two — Value ticks downstream, Quiet does neither, and Silent updates the value slot WITHOUT ticking, which is exactly what delay needs and the reason a "did it fire?" boolean is not enough. That was settled as a contract question up front rather than discovered while porting delay.



The node changed completely. The wiring barely moved.

The payoff slide for the whole rewrite argument, and the one that answers "did your users have to rewrite everything?" — no. Roughly 90 seconds. Top row: the node source is unrecognisable. `&mut self` with stored upstreams and RefCell fields becomes four associated types and a free function over explicit arguments. That is the inversion, in source. Bottom row: the wiring is one extra line. You hold a GraphBuilder and build sources on it instead of calling free functions. That is the entire user-facing cost of the change. If asked what else moved: `Rc<dyn Stream<T>>` became `Stream<T>`, a cheap Clone handle, and operator traits became extension traits. RunMode, RunFor and NanoTime kept their shape — both engines share one runtime core.



Two engines, one copy of the semantics

Be precise here, because this is where an overclaim would get caught. Tier has two variants — Interpreted and Compiled — and WINGFOIL_TIER picks between them. The island is the compiled emission with an interpreted boundary, not a third thing. The seam is the part worth selling: you do not choose globally. On the three-node graph an island is a dead wash with the interpreter (1.40 ms vs 1.43 ms) because a boundary needs something to amortise against; on the bigger graphs it lands 2.2–10.2× ahead. That spread is what "it is the seam, not an engine" looks like in the numbers. If you ever find yourself writing a second copy of what a node does, that is the invariant breaking — the only one in the codebase treated as non-negotiable.

The node changed completely. The wiring barely moved.

WHAT IT BUYS

Two engines, one copy of the semantics

Engine	Who owns state	Dispatch
Interpreted	the builder's slots	one dyn call per node
Compiled	locals in a generated fn	monomorphized, inlinable

One implementation — which is why they cannot drift.

...and you choose the boundary, not the engine

A **nested island** mounts the compiled emission as *one node* of an interpreted graph. Not a third engine — which is why it lands between the two.

You write the wiring once, in plain Rust

The constraint worth stating: wiring must be straight-line. Values and per-element logic can be as procedural as you like — the TOPOLOGY cannot depend on runtime values, because compiled() monomorphizes one local per node and the node list has to be complete after parsing. If asked about loops: .map_n / .fan with a literal count.

THE FRONT DOOR

You write the wiring once, in plain Rust

```
wingfoil::mitrol {
  fn odds_events(g: &graphBuilder) -> Stream<String> {
    let count = g.ticker(PERIOD).count();
    let is_even = count.map(|i| i.is_multiple_of(2));
    let is_odd = is_even.map(|b| !b);
    let odd_str = count.filter(&is_odd).map(|i| format!("{i} is odd"));
    let even_str = count.filter(&is_even).map(|i| format!("{i} is even"));
    let labelled = odd_str.merge(even_str);
    labelled
  }
}
```

Out comes `odds_events::interpreted()`, `odds_events::compiled()` and `odds_events::nested()` — from the same tokens.

The wiring function stays **valid, reviewable Rust** either way. That property was deliberately never traded away. `count` is read three times, so it is a shared apex node — the macro derives a real DAG, not a chain.

One function. One local per node. No dyn.

Point at the call: that is the SAME cycle function the interpreter calls. Not a re-implementation — a call. That is wall 1 being dead. Point at ``context("node 0 (wf_anon_1: ticker) cycle")`` — error context uses the binding name the USER wrote, baked in as a `&'static str` at expansion time, so it costs nothing until something fails. The full expansion is committed in the repo so nobody has to trust me.

...AND THIS IS WHAT IT BECOMES

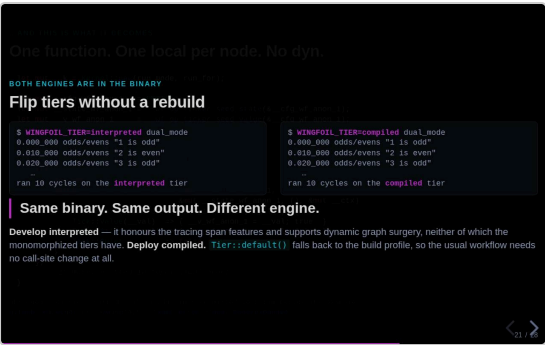
One function. One local per node. No dyn.

```
let mut __k = Kernel::new(run_mode, run_for);

let mut __cfg_wf_anon_1 = PERIOD;
let mut __state_wf_anon_1 = __wf_op_ticker_seed_state(&__cfg_wf_anon_1);
let mut __v_wf_anon_1 = __wf_op_ticker_seed_value(&__cfg_wf_anon_1);
// new cfg / state / value local per node, 11 nodes ...

let mut __dirty = [false; 11usize];
while __k.loop_cycle(&mut __dirty) {
  let __t_wf_anon_1 = __activation / && {
    let mut __ctx = __ctx::new(&mut __A, &mut __B);
    match __wf_op_ticker_cycle(&mut __cfg_wf_anon_1, &mut __state_wf_anon_1, (), &mut __ctx) {
      .context("node 0 (wf_anon_1: ticker) cycle")? {
        Tick::Value(__val) => { __v_wf_anon_1 = __val; true }
        Tick::Silent(__val) => { __v_wf_anon_1 = __val; false }
        Tick::Quiet => false,
      }
    }
  };
  // 10 more, emitted in topological order ...
}
```

stripped - ellipses marked - all 1,730 lines committed verbatim beside the example:
[github.com/wingfoil-io/wingfoil - examples/core/dual_mode/expanded/](https://github.com/wingfoil-io/wingfoil/blob/master/examples/core/dual_mode/expanded/)



21 / 28

BOTH ENGINES ARE IN THE BINARY

Flip tiers without a rebuild

That the tiers AGREE is not a hope — tests/macro_parity.rs runs this same diamond through both engines and pins the outputs equal. That test is the structural form of the invariant: the property stays true rather than merely intended.

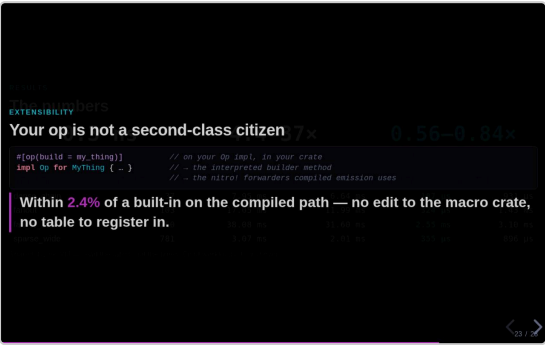


22 / 28

RESULTS

The numbers

The bar we set: the new INTERPRETED engine had to be at least as fast as the old one, because most graphs keep running interpreted and a rewrite that trades everyday performance for a fast path nobody uses is a bad trade. It came in faster on all eight. Islands land 2.2–10.2×, except on a three-node graph where they are a wash — an island needs something to amortise its boundary against.

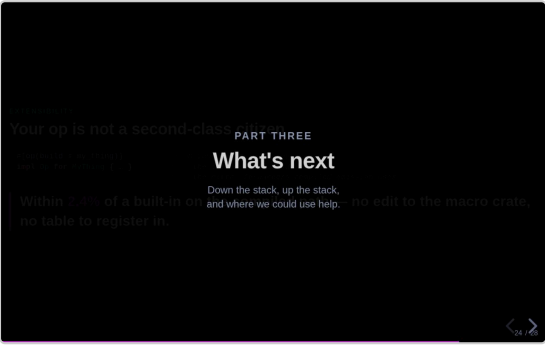


23 / 28

EXTENSIBILITY

Your op is not a second-class citizen

"That was the property we most doubted was achievable, and it is the one that most changes what the library is for." A library where third-party ops are second-class is a library you eventually fork. This one you extend.

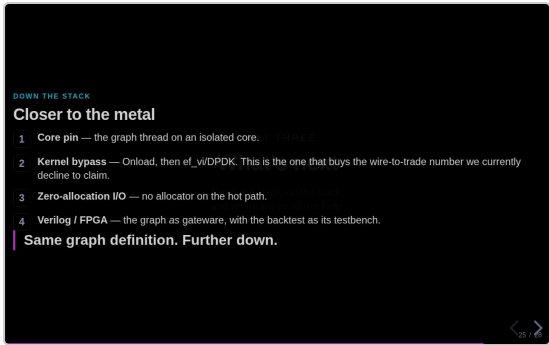


24 / 28

PART THREE

What's next

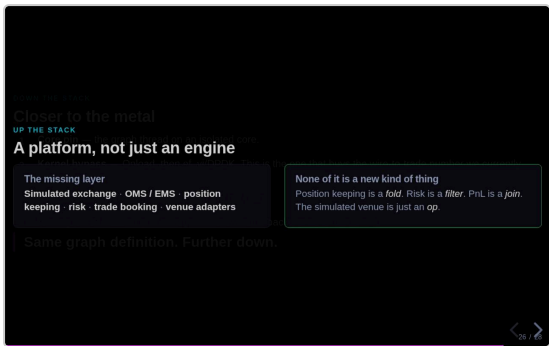
Transition, ~19 minutes in. From here it is all forward-looking — change your energy: less explaining, more inviting.



25 / 28
DOWN THE STACK

Closer to the metal

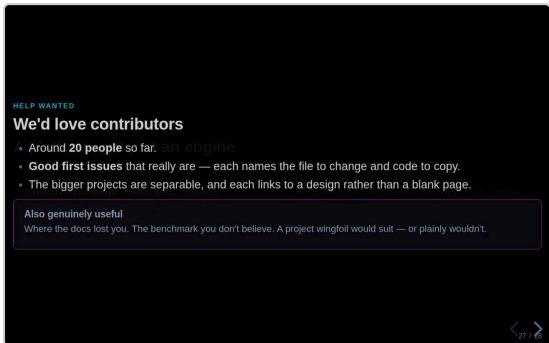
~75 seconds. The framing that makes this land: every one of these is the same graph definition, further down. Nothing about your wiring changes as you descend — that is the whole payoff of part two. FPGA is the exploratory one. Say so; do not oversell it.



26 / 28
UP THE STACK

A platform, not just an engine

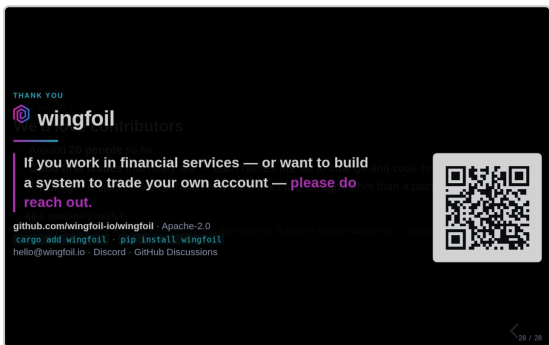
~90 seconds, and this is the slide the traders in the room care about. Be straight that wingfoil is a compute engine today, not a platform, and that the aim is the ~20% a serious latency-focused shop actually needs rather than parity with an incumbent's breadth. The right-hand card is the argument for why this is tractable at all.



27 / 28
HELP WANTED

We'd love contributors

~60 seconds. Be specific rather than gracious — "good first issues" is a label everyone has been burned by, so the sentence that matters is that each one names the file and an example to copy. Check the contributor count before you say it; it is the one number on these slides I could not verify from the repo, because the clone is shallow.



28 / 28
THANK YOU

wingfoil

Land the ask and stop talking. Leave this up for Q&A — the QR is the reason to leave it up rather than black the screen. If the room is quiet, prompt with: "the question I usually get is what happens when the graph shape isn't known until runtime — that's Project Lightning, and it's open."